

Assignment 10: Data Scraping

Evan Lim

OVERVIEW

This exercise accompanies the lessons in Environmental Data Analytics on data scraping.

Directions

1. Rename this file `<FirstLast>_A10_DataScraping.Rmd` (replacing `<FirstLast>` with your first and last name).
2. Change “Student Name” on line 3 (above) with your name.
3. Work through the steps, **creating code and output** that fulfill each instruction.
4. Be sure your code is tidy; use line breaks to ensure your code fits in the knitted output.
5. Be sure to **answer the questions** in this assignment document.
6. When you have completed the assignment, **Knit** the text and code into a single PDF file.

Set up

1. Set up your session:
 - Load the packages `tidyverse`, `rvest`, and any others you end up using.
 - Check your working directory

```
#1 Loading packages
library(tidyverse)
library(rvest)
library(here)

# Checking working directory
here()
```

```
## [1] "/home/guest/EDE_Fall2025"
```

2. We will be scraping data from the NC DEQs Local Water Supply Planning website, specifically the Durham’s 2024 Municipal Local Water Supply Plan (LWSP):
 - Navigate to <https://www.ncwater.org/WUDC/app/LWSP/search.php>
 - Scroll down and select the LWSP link next to Durham Municipality.
 - Note the web address: <https://www.ncwater.org/WUDC/app/LWSP/report.php?pwsid=03-32-010&year=2024>

Indicate this website as the as the URL to be scraped. (In other words, read the contents into an `rvest` webpage object.)

#2 Setting the website to scrape from

```
durham_lwsp <- read_html(  
  "https://www.ncwater.org/WUDC/app/LWSP/report.php?pwsid=03-32-010&year=2024")
```

3. The data we want to collect are listed below:

- From the “1. System Information” section:
- Water system name
- PWSID
- Ownership
- From the “3. Water Supply Sources” section:
- Maximum Day Use (MGD) - for each month

In the code chunk below scrape these values, assigning them to four separate variables.

HINT: The first value should be “Durham”, the second “03-32-010”, the third “Municipality”, and the last should be a vector of 12 numeric values (represented as strings)“.

#3 Scraping data

```
system_name <- durham_lwsp %>%  
  html_nodes('div+ table tr:nth-child(1) td:nth-child(2)') %>%  
  html_text()  
  
pwsid <- durham_lwsp %>%  
  html_nodes('td tr:nth-child(1) td:nth-child(5)') %>%  
  html_text()  
  
ownership <- durham_lwsp %>%  
  html_nodes('div+ table tr:nth-child(2) td:nth-child(4)') %>%  
  html_text()  
  
mgd <- durham_lwsp %>%  
  html_nodes('th~ td+ td') %>%  
  html_text()
```

4. Convert your scraped data into a dataframe. This dataframe should have a column for each of the 4 variables scraped and a row for the month corresponding to the withdrawal data. Also add a Date column that includes your month and year in data format. (Feel free to add a Year column too, if you wish.)

TIP: Use `rep()` to repeat a value when creating a dataframe.

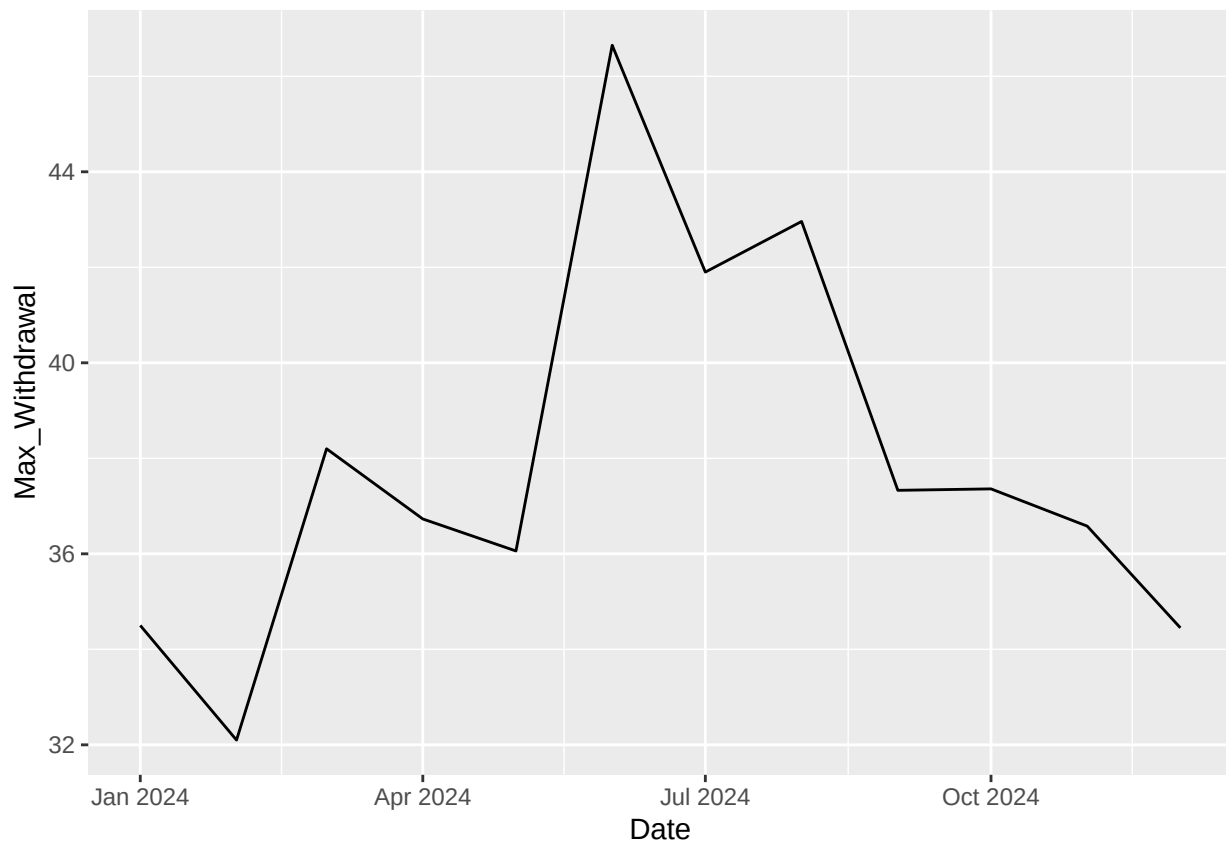
NOTE: It’s likely you won’t be able to scrape the monthly withdrawal data in chronological order. You can overcome this by creating a month column manually assigning values in the order the data are scraped: “Jan”, “May”, “Sept”, “Feb”, etc. . .

5. Create a line plot of the maximum daily withdrawals across the months for 2024, making sure, the months are presented in proper sequence.

```
#4 Creating a data frame
durham_mgd_2024 <- data.frame("Month" = c("Jan", "May", "Sep",
                                           "Feb", "Jun", "Oct",
                                           "Mar", "Jul", "Nov",
                                           "Apr", "Aug", "Dec"),
                              "Year" = rep(2024,12),
                              "Max-Withdrawal" = as.numeric(mgd))

# Adding columns and arranging by date
durham_mgd_2024 <- durham_mgd_2024 %>%
  mutate(System_Name = !!system_name,
         PWSID = !!pwsid,
         Ownership = !!ownership,
         Date = my(paste(Month, "-", Year))) %>%
  arrange(Date)

#5
ggplot(durham_mgd_2024,
       aes(x = Date,
           y = Max-Withdrawal
           )) +
  geom_line()
```



6. Note that the PWSID and the year appear in the web address for the page we scraped. Construct a function with two inputs - “PWSID” and “year” - that:

- Creates a URL pointing to the LWSP for that PWSID for the given year
- Creates a website object and scrapes the data from that object (just as you did above)
- Constructs a dataframe from the scraped data, mostly as you did above, but includes the PWSID and year provided as function inputs in the dataframe.
- Returns the dataframe as the function’s output

#6. Creating a scraping function

```
scrape.it <- function(the_pwsid, the_year){
  the_website <- read_html(paste0(
    'https://www.ncwater.org/WUDC/app/LWSP/report.php?pwsid=',
    the_pwsid,
    "&year=",
    the_year))

  system_name <- the_website %>% html_nodes('div+ table tr:nth-child(1) td:nth-child(2)') %>% html_text()
  ownership <- the_website %>% html_nodes('div+ table tr:nth-child(2) td:nth-child(4)') %>% html_text()
  mgd <- the_website %>% html_nodes('th~ td+ td') %>% html_text()

  df_withdrawals <- data.frame("Month" = c("Jan", "May", "Sep",
                                           "Feb", "Jun", "Oct",
                                           "Mar", "Jul", "Nov",
                                           "Apr", "Aug", "Dec"),
                              "Year" = rep(the_year,12),
                              "PWSID" = rep(the_pwsid, 12),
                              "Max-Withdrawal" = as.numeric(mgd)) %>%
    mutate(System_Name = !!system_name,
           Ownership = !!ownership,
           Date = my(paste(Month,"-",Year))) %>%
    arrange(Date)

  return(df_withdrawals)
}
```

7. Use the function above to extract and plot max daily withdrawals for Durham (PWSID=‘03-32-010’) for each month in 2020. Then show the structure of the dataframe with either the `str()` or the `glimpse()` function.

#7 Applying function for Durham 2020

```
durham_2020 <- scrape.it('03-32-010', 2020)
```

#showing structure of dataframe

```
str(durham_2020)
```

```
## 'data.frame':   12 obs. of  7 variables:
## $ Month       : chr  "Jan" "Feb" "Mar" "Apr" ...
## $ Year        : num  2020 2020 2020 2020 2020 2020 2020 2020 2020 2020 ...
## $ PWSID       : chr  "03-32-010" "03-32-010" "03-32-010" "03-32-010" ...
## $ Max-Withdrawal: num  36 32 37.3 32.4 37 ...
## $ System_Name  : chr  "Durham" "Durham" "Durham" "Durham" ...
```

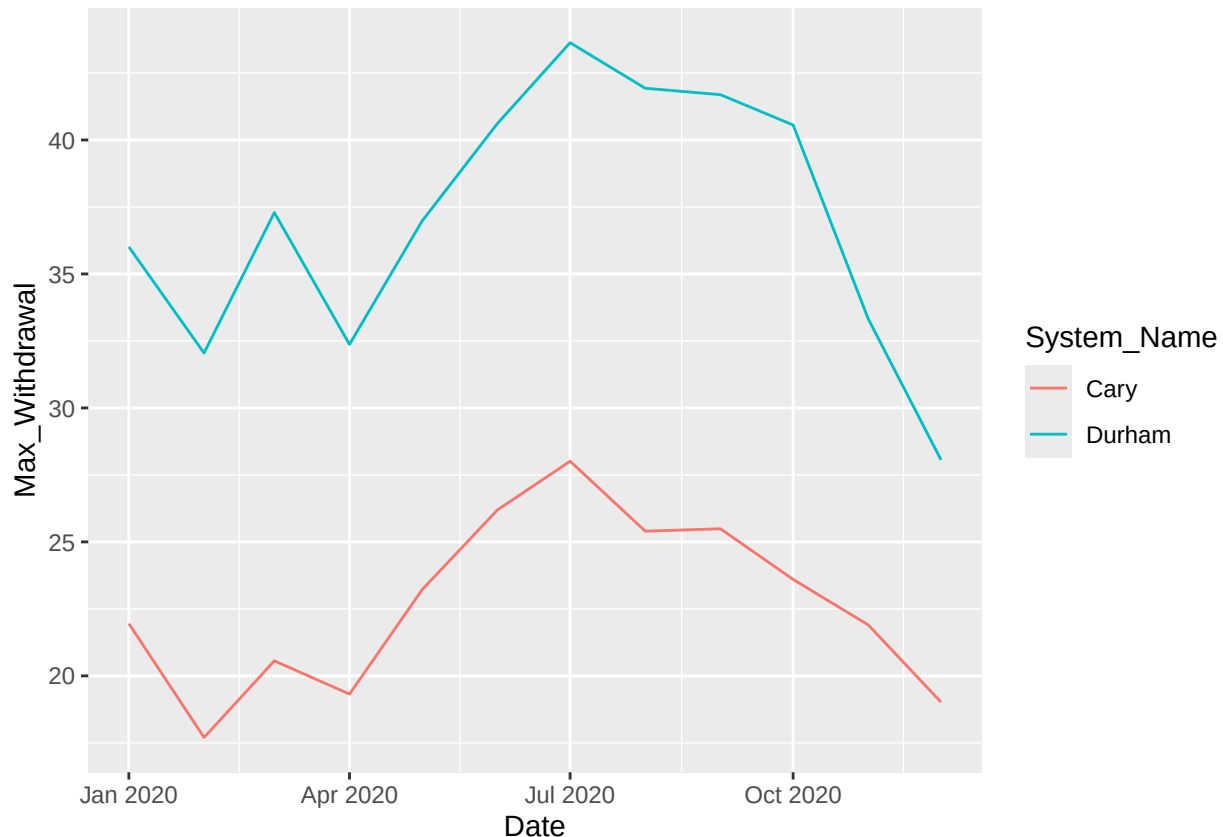
```
## $ Ownership      : chr "Municipality" "Municipality" "Municipality" "Municipality" ...
## $ Date           : Date, format: "2020-01-01" "2020-02-01" ...
```

- Use the function above to extract data for Cary (PWSID = '03-32-010') in 2020. Combine this data with the Durham data collected above and create a plot that compares Cary's to Durham's water withdrawals.

```
#8 Applying function for Cary 2020
cary_2020 <- scrape.it('03-32-010', 2020)

#Binding Cary and Durham 2020 data
cary_durham_2020 <- rbind(cary_2020, durham_2020)

ggplot(cary_durham_2020,
  aes(x = Date,
    y = Max-Withdrawal,
    color = System_Name)) +
  geom_line()
```



- Use the code & function you created above to plot Cary's max daily withdrawal by months for the years 2018 thru 2023. Add a smoothed line to the plot (method = 'loess').

TIP: See Section 3.2 in the "10_Data_Scraping.Rmd" where we apply "map2()" to iteratively run a function over two inputs. Pipe the output of the map2() function to bind_rows() to combine the dataframes into a single one, and use that to construct your plot.

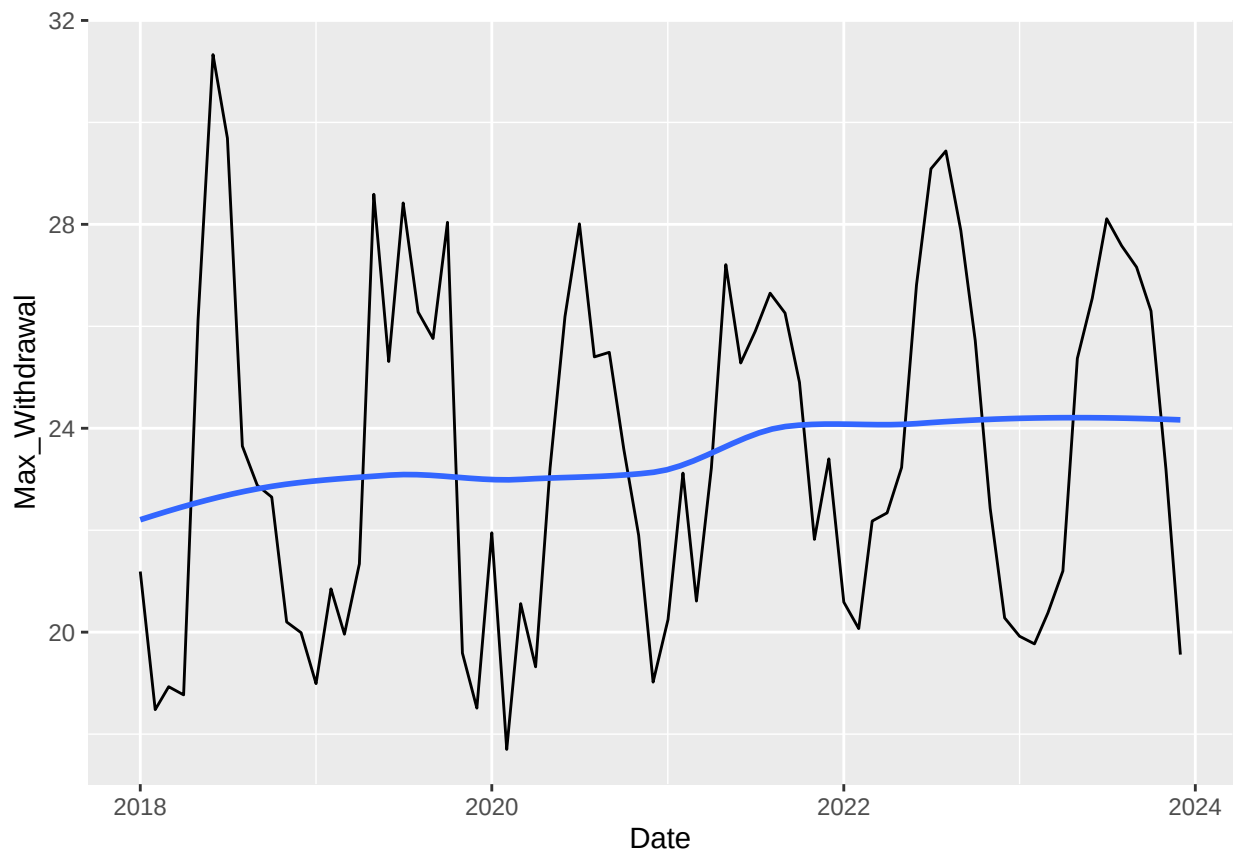
```
#9
the_years <- 2018:2023

cary_2018_2023_separate <- map2('03-92-020', the_years, scrape.it)

cary_2018_2023_combined <- cary_2018_2023_separate %>%
  bind_rows()

ggplot(cary_2018_2023_combined,
  aes(x = Date,
      y = Max-Withdrawal)) +
  geom_line() +
  geom_smooth(method = 'loess', se = FALSE)

## 'geom_smooth()' using formula = 'y ~ x'
```



Question: Just by looking at the plot (i.e. not running statistics), does Cary have a trend in water usage over time? > Answer: Just by looking at the plot, Cary appears to have a trend of increasing water usage over time from 2018 to 2023. While we will need more data to tell, things do appear to plateau starting from 2022, after having risen fairly steadily since 2018.